

---

**robotat**

**ChristianCruz**

**23 de octubre de 2025**



|                                          |           |
|------------------------------------------|-----------|
| <b>1. Configuracion</b>                  | <b>3</b>  |
| 1.1. crazyflies.yaml . . . . .           | 3         |
| 1.2. motion_capture.yaml . . . . .       | 3         |
| 1.3. server.yaml . . . . .               | 4         |
| 1.4. Modulos Crazyradio PA . . . . .     | 5         |
| <b>2. Comandos</b>                       | <b>7</b>  |
| 2.1. Crazyflie Server . . . . .          | 7         |
| 2.2. Rutinas individuales . . . . .      | 8         |
| 2.3. Rutinas de enjambre . . . . .       | 8         |
| 2.4. cfGui.py . . . . .                  | 8         |
| <b>3. Rutinas de vuelo individuales</b>  | <b>9</b>  |
| 3.1. hover . . . . .                     | 9         |
| 3.2. goToInitialPosition . . . . .       | 9         |
| 3.3. goToOrigin . . . . .                | 10        |
| 3.4. goToRigidBody . . . . .             | 10        |
| 3.5. goToChargingStation . . . . .       | 10        |
| 3.6. followRigidBody . . . . .           | 10        |
| 3.7. Otros nodos . . . . .               | 10        |
| 3.8. Creacion de rutinas . . . . .       | 10        |
| <b>4. Rutinas de vuelo en enjambre</b>   | <b>11</b> |
| 4.1. multi_hover . . . . .               | 11        |
| 4.2. multi_goToInitialPosition . . . . . | 11        |
| 4.3. multi_goToRigidBody . . . . .       | 11        |
| 4.4. multi_followRigidBody . . . . .     | 11        |



En este proyecto se desarrolla la implementacion de un enjambre de drones Crazyflie 2.1 en el ecosistema robotico Robotat en la Universidad del Valle de Guatemala mediante Crazyswarm2 y ROS2. Crazyswarm2 es una paquete de ROS2 creado para controlar los agentes roboticos de Bitcraze, es un proyecto de codigo abierto y disponible en el siguiente [github](#). El siguiente articulo describe el proyecto original de Crazyswarm y es la base de todo el proyecto.

```
@inproceedings{crazyswarm,
author    = {James A. Preiss* and
             Wolfgang H\"onig* and
             Gaurav S. Sukhatme and
             Nora Ayanian},
title     = {Crazyswarm: {A} large nano-quadcopter swarm},
booktitle = {{IEEE} International Conference on Robotics and Automation (
↪{ICRA})},
pages     = {3299--3304},
publisher = {{IEEE}},
year      = {2017},
url       = {https://doi.org/10.1109/ICRA.2017.7989376},
doi       = {10.1109/ICRA.2017.7989376},
note     = {Software available at \url{https://github.com/USC-ACTLab/
↪crazyswarm}},
}
```

En esta documentacion se describen los pasos para la configuracion de Crazyswarm2 en nuestro entorno, asi como las rutinas de vuelo individuales y de enjambre desarrolladas para el proyecto. Ademas, se incluyen recomendaciones de uso y consideraciones importantes para el correcto funcionamiento del sistema.



### 1.1 crazyflies.yaml

El archivo `crazyflies.yaml` contiene la configuración específica para los drones Crazyflie utilizados en el sistema. Este archivo define parámetros como la dirección del radio, posiciones iniciales y tipo de robot. En este proyecto se utilizan diferentes números de antena y canales de radio para evitar interferencias entre los drones, y el mismo tipo de drone (`cf21_single_marker`) para todos ellos.

```
robots:
  cf0:
    enabled: false                # Habilitar o deshabilitar este
    ↪drone
    uri: radio://0/80/2M/E7E7E7E7EA # URI "radio:// Numero de
    ↪Crayradio // Canal / Velocidad de datos / Dirección del Crazyflie"
    initial_position: [0.0, 0.0, 0.0] # Posición inicial (x, y, z)
    type: cf21_single_marker        # Tipo de robot
```

El tipo de robot también se define en `crazyflies.yaml` y hace referencia a una configuración específica, para este proyecto se utiliza el tipo `cf21_single_marker`, el cual se encuentra definido en `motion_capture.yaml`, usando **librigidbodytracker** para el tracking de los drones en lugar de usar el tracker específico del fabricante, en nuestro caso Motive.

```
robots_types:
  cf21_single_marker:
    motion_capture:
      tracking: "librigidbodytracker" # one of "vendor",
    ↪"librigidbodytracker"
      marker: cf_single_marker
      dynamics: default
      big_quad: false
      battery:
```

(continúe en la próxima página)

```
voltage_warning: 3.8 # V
voltage_critical: 3.7 # V
```

En este archivo tambien se definen configuraciones globales para todos los drones, como la frecuencia de actualización de la posicion y estado, topicos personalizados, tipo de controlador y estimador, etc.

## 1.2 motion\_capture.yaml

El archivo `motion_capture.yaml` contiene la configuración del sistema de captura de movimiento utilizado para rastrear la posición y orientación de los drones en el espacio. Este archivo define parámetros como la frecuencia de actualización, el tipo de sistema de captura de movimiento y otros ajustes relacionados con la integración del sistema de captura de movimiento con CrazySwarm2. Para este proyecto se cuenta con las siguientes configuraciones:

- **Tipo:** `optitrack_closed_source`, esta configuracion representa al sistema de captura de movimiento OptiTrack, utilizando las versiones mas recientes del software Motive.
- **Hostname:** esta configuracion define la direccion IP del computador donde se esta ejecutando el software de captura de movimiento, segun la infraestructura del Robotat esta direccion es la direccion del router ubicado en el laboratorio.

```
ros__parameters:
  # one of "optitrack", "optitrack_closed_source", "vicon", "qualisys",
  ↪ "nokov", "vrpn", "motionanalysis"
  type: "optitrack_closed_source"
  # Specify the hostname or IP of the computer running the motion capture_
  ↪ software
  hostname: "192.168.50.200"
  # mode: "motionCapture"
  topics:
  poses:
    qos:
    mode: "sensor"
    deadline: 100.0 # Hz
```

En este mismo archivo se definen las configuraciones de los marcadores utilizados para el tracking de los drones. En este proyecto se utiliza un marcador por dron, el cual esta definido como `cf_single_marker`, y cuenta con un offset en z para considerar la altura del marcador respecto al centro del dron.

```
marker_configurations:
  cf_single_marker:
    offset: [0.0, 0.0, 0.0]
    points:
      p0: [0.0, 0.0, 0.023]
```

Por ultimo se definen las configuraciones de dinamica del dron, las cuales son utilizadas por el controlador para limitar la velocidad y aceleracion del dron. En este proyecto se utiliza la configuracion default, la cual define limites de velocidad y aceleracion adecuados para los drones Crazyflie 2.1.



```
dynamics_configurations:
  default:
    max_velocity: [1.5, 1.5, 1.0]
    max_angular_velocity: [7, 7, 5]
    max_roll: 0.7 # 40°
    max_pitch: 0.7 # 40°
    max_fitness_score: 0.001
```

## 1.3 server.yaml

En el archivo `server.yaml` se definen configuraciones para las alertas del Crazyflie Server y configuraciones relacionadas con la simulacion. En este caso solo se modifica la frecuencia de las alertas para el sistema de captura de movimiento y la comunicacion con los drones.

```
ros__parameters:
  warnings:
    frequency: 10.0 # report/run checks once per second
  motion_capture:
    warning_if_rate_outside: [80.0, 120.0]
  communication:
    max_unicast_latency: 30.0 # ms
    min_unicast_ack_rate: 0.9
    min_unicast_receive_rate: 0.9 # requires status topic to be enabled
    min_broadcast_receive_rate: 0.9 # requires status topic to be enabled
    publish_stats: false
```

## 1.4 Modulos Crazyradio PA

Para la comunicacion entre el computador y los drones Crazyflie se utilizan modulos Crazyradio PA, los cuales permiten una comunicacion estable y de largo alcance. Estos modulos se conectan al computador por medio de un puerto USB y se configuran en el archivo `crazyflies.yaml`. Para poder utilizar los Crazyradio PA es necesario seguir los pasos de instalacion y configuracion descritos en la documentacion oficial de Bitcraze, disponibles en el siguiente enlace: [Crazyradio PA Installation and USB Permissions](#).

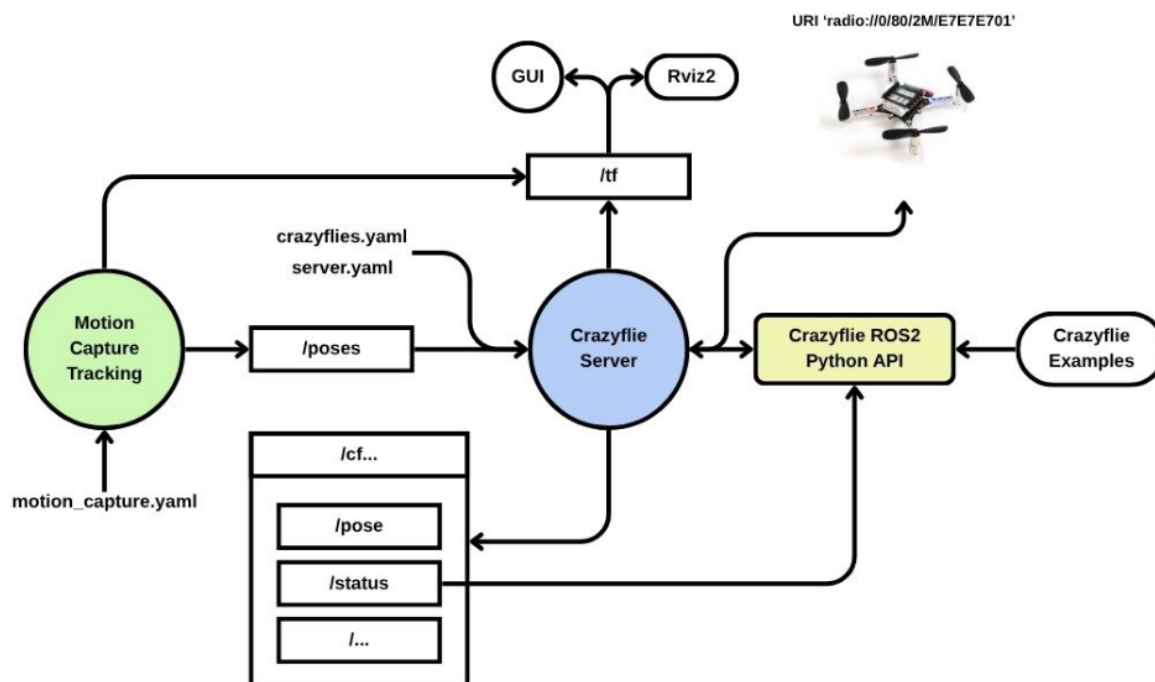
Ademas de seguir estos pasos es necesario actualizar el firmware de la Crazyradio PA al incluido en Crazyswarm2, el cual se encuentra en la carpeta `/ros2_ws/src/crazyswarm2/prebuilt`. Los pasos para actualizar el firmware se encuentran en la documentacion oficial de Bitcraze, disponibles en el siguiente enlace: [Updating the Crazyradio PA firmware](#).



Crazyswarm2 es un paquete de ROS2, por lo cual, todas las herramientas y comandos se ejecutan por medio de la terminal. A continuación se describen los principales comandos utilizados para ejecutar Crazyswarm2.

### 2.1 Crazyflie Server

El comando `ros2 launch crazyflie launch.py backend:=cflib` inicia el servidor de Crazyswarm2, que es responsable de gestionar la comunicación con los drones Crazyflie. Este comando debe ejecutarse antes de iniciar cualquier rutina de vuelo o comando relacionado con los drones. En este caso se utiliza el backend `cflib`, que es el backend de python que presento mejores resultados en el proyecto.



Se pueden modificar los argumentos predeterminados del Crazyflie Server, como el backend utilizado o los nodos lanzados al momento de iniciar el servidor. Esto se puede realizar agregando argumentos al comando de lanzamiento o modificando directamente el archivo launch ubicado en `/ros2_ws/src/crazyswarm2/crazyflie/launch`. Para este proyecto, se modifico el archivo launch para deshabilitar el nodo de **gui** y no mostrar la interfaz grafica predeterminada de Crazyswarm2, ya que se desarrollo una interfaz grafica personalizada para el proyecto.

## 2.2 Rutinas individuales

Para ejecutar las rutinas individuales, se utiliza el comando `ros2 run robotat <nombre_rutina>`. Cada rutina es un nodo independiente de ROS2 que puede ser ejecutado por separado. Para incluir un parametro al momento de ejecutar el nodo, se utiliza la siguiente sintaxis: `ros2 run robotat <nombre_rutina> --ros-args -p <parametro1>:=<valor1> -p <parametro2>:=<valor2>`. Los parametros disponibles para cada rutina se describen en la documentación de las rutinas.

## 2.3 Rutinas de enjambre

Para ejecutar las rutinas de enjambre, se utiliza el comando `ros2 launch robotat <nombre_rutina_enjambre>.launch.py`. Cada rutina de enjambre es un archivo launch que inicia multiples nodos de rutina individual, uno por cada dron en el enjambre. Los parametros para cada nodo se definen en el archivo `crazyflies_robotat.yaml`.

## 2.4 cfGui.py

La interfaz grafica no se inicia como un nodo de ROS2, sino que se ejecuta por separado. Para iniciar la interfaz grafica, se utiliza el comando `python3 cfGui.py` desde la carpeta robotat o desde VsCode.

Esta interfaz permite monitorear el estado de los drones y visualizar la posición de los drones en tiempo real.



---

### Rutinas de vuelo individuales

---

Las rutinas de vuelo individuales son secuencias predefinidas que un solo dron puede ejecutar de manera autónoma. Estas rutinas permiten realizar tareas específicas, como despegar, aterrizar, mantener una posición fija, moverse a una posición inicial, entre otras.

Cada rutina fue desarrollada como un nodo independiente de ROS2, lo que permite leer los topicos de estado y posicion relacionadas a cada Crazyflie, y obtener parametros al momento de ejecutar cada nodo. Dentro de los parametros definidos se incluyen el numero de Crazyflie (`cf_number`), offsets (`offset`), nombre de cuerpo rigido (`rigid_body_name`), entre otros.

Todas las rutinas cuentan con una estructura similar, que incluye la inicialización del nodo, la suscripción a los topicos necesarios, la configuración de los parametros y la implementación de la lógica de la rutina. La logica incluye la obtencion del nivel de bateria para verificar que el vuelo se pueda realizar, la obtencion de la posicion actual del dron y meta, calculo de distancia y tiempo adecuado para el movimiento, finalizando con envio de comandos de control para ejecutar la rutina deseada.

#### Recomendaciones de uso

- Antes de ejecutar cualquier rutina, asegúrese de que el Crazyflie Server esté en funcionamiento y que el dron esté correctamente configurado en el archivo `crazyflies.yaml`.
- Alinear el eje x del dron con el eje x del sistema de captura de movimiento para evitar errores en la orientacion del dron.
- Verificar que la batería del dron tenga un nivel adecuado antes de iniciar.

A continuación se describen las principales rutinas de vuelo individuales desarrolladas:

### 3.1 hover

La rutina `hover` permite que el dron mantenga una posición fija en el espacio. Al ejecutar este nodo, el dron despegue y se posiciona en una altura predeterminada en el mismo lugar donde se encuentra, manteniéndose estable en esa posición durante un tiempo. Esta rutina es útil para verificar el funcionamiento del dron y Crazyswarm2.

## 3.2 goToInitialPosition

La rutina `goToInitialPosition` permite que el dron se desplace a su posicion inicial definida en `crazyflies.yaml`. Al ejecutar este nodo, el dron despegue y vuela hacia su posicion inicial. Esta rutina se utilizo para reiniciar la flota de drones pues estos deben encontrarse en su posicion inicial para iniciar el **Crazyflie Server**.

## 3.3 goToOrigin

La rutina `goToOrigin` permite que el dron se desplace a la posicion  $[0,0,0]$ . Esta rutina es util para centrar la posicion del dron en el espacio de vuelo.

## 3.4 goToRigidBody

La rutina `goToRigidBody` permite que el dron se desplace a la posicion de un cuerpo rigido especifico. Al ejecutar este nodo, el dron vuela hacia la posicion del cuerpo rigido definido en los parametros. Esta rutina es util para posicionar el dron en una ubicacion especifica.

## 3.5 goToChargingStation

La rutina `goToChargingStation` permite que el dron se desplace a la posicion de una estacion de carga. Al ejecutar este nodo, el dron vuela hacia la posicion de la estacion de carga. Esta rutina es util para llevar el dron a una ubicacion donde pueda recargar su bateria.

## 3.6 followRigidBody

La rutina `followRigidBody` permite que el dron siga a un cuerpo rigido especifico. Al ejecutar este nodo, el dron ajusta su posicion en tiempo real para mantenerse cerca del cuerpo rigido definido en los parametros. Esta rutina es util para realizar seguimientos de objetos en movimiento.

## 3.7 Otros nodos

- **graph**. Este nodo se utiliza para obtener la posicion de objetos en el espacio de vuelo y graficar su trayectoria durante el tiempo que el nodo estuvo activo, guardando los datos en un archivo csv.
- **kalman**. Este nodo compara la posicion medida por el sistema de captura de movimiento y la posicion estimada por el filtro de Kalman de un dron individual, graficando ambas posiciones para analizar la precision del filtro.
- **latencyMeasure**. Este nodo mide la latencia entre **Crazyswarm2** y los **Crazyflies**, guardando los datos en un archivo csv para su posterior analisis.



## 3.8 Creacion de rutinas

Para crear una nueva rutina de vuelo individual, se puede tomar como referencia alguna de las rutinas existentes. Es importante seguir la estructura y convenciones utilizadas en las rutinas actuales para asegurar la compatibilidad con CrazySwarm2 y ROS2. Se deben seguir el mismo procedimiento para crear un nodo dado en la documentacion de ROS2.



---

### Rutinas de vuelo en enjambre

---

Se modificaron las rutinas individuales para poder controlar mas de un solo dron, esto por medio de ciclos que envíen los mismos comandos a los drones seleccionados. Cada dron cuenta con diferentes parametros, los cuales se definen en el archivo `crazyflies_robotat.yaml`. Las rutinas desarrolladas fueron funciones simples para verificar el correcto funcionamiento de Crazyswarm2, dejando espacio a implementar rutinas mas complejas en el futuro.

#### 4.1 multi\_hover

La rutina `multi_hover` permite que todos los drones en el enjambre mantengan una posición fija en el espacio. Al ejecutar este nodo, cada dron despegue y se posiciona en una altura predeterminada en el mismo lugar donde se encuentra, manteniéndose estable en esa posición durante un tiempo.

#### 4.2 multi\_goToInitialPosition

La rutina `multi_goToInitialPosition` permite que todos los drones en el enjambre se desplacen a su posición inicial definida en `crazyflies.yaml`. Al ejecutar este nodo, cada dron despegue y vuela hacia su posición inicial. Esta rutina se utilizó para reiniciar la flota de drones pues estos deben encontrarse en su posición inicial para iniciar el **Crazyflie Server**.

#### 4.3 multi\_goToRigidBody

La rutina `multi_goToRigidBody` permite que todos los drones en el enjambre se desplacen a la posición de un cuerpo rígido específico. Al ejecutar este nodo, cada dron vuela hacia la posición del cuerpo rígido definido con un offset definido en los parametros.

## 4.4 multi\_followRigidBody

La rutina `multi_followRigidBody` permite que todos los drones en el enjambre sigan a un cuerpo rígido específico. Al ejecutar este nodo, cada dron ajusta su posición en tiempo real para mantenerse cerca del cuerpo rígido definido con un offset definido en los parámetros.